

De la propagation avant à la rétropropagation : une analyse des mécanismes d'apprentissage dans les réseaux de neurones multicouches

Sasha Sutton¹

¹ Aix-Marseille Université, France, sasha.sutton@etu.univ-amu.fr

Résumé

Cet article présente les fondements mathématiques et techniques des réseaux de neurones artificiels. À partir du modèle de neurone simple introduit par Rosenblatt, nous décrivons le perceptron et ses limites face aux problèmes non linéairement séparables. Nous introduisons ensuite les réseaux de neurones multicouches, dont la capacité d'approximation universelle est établie théoriquement. L'algorithme de rétropropagation du gradient, mécanisme central de l'apprentissage, est présenté en détail. Enfin, les principaux problèmes pratiques rencontrés lors de l'entraînement des réseaux de neurones sont discutés, notamment le surapprentissage et les méthodes de régularisation, ainsi que les solutions associées.

Mots clés réseau de neurones multicouches ; propagation avant ; rétropropagation ; descente de gradient ; apprentissage automatique

1 Introduction

Les réseaux de neurones artificiels sont devenus un outil central de l'apprentissage automatique. Leurs applications, répertoriées notamment par Goodfellow et al. [1], couvrent des domaines variés tels que le diagnostic médical par imagerie, la conduite autonome ou la traduction automatique. Sur certaines tâches de classification visuelle, He et al. [2] montrent que leurs résultats dépassent même ceux des experts humains. Cette efficacité soulève une question sur les mécanismes mathématiques qui permettent à ces systèmes d'apprendre. Ces réseaux présentent aussi des faiblesses que leurs bons résultats ne font pas toujours apparaître. Moosavi-Dezfooli et al. [3] montrent qu'il existe de petites perturbations imperceptibles à l'œil humain qui suffisent à tromper ces réseaux sur la quasi-totalité des images naturelles. Pour comprendre et corriger de telles faiblesses, il faut d'abord analyser précisément comment ces réseaux apprennent. C'est l'objectif de cet article.

Historiquement, les premières formalisations remontent aux années 1940, avec le modèle de neurone formel proposé par McCulloch et Pitts [4], puis au perceptron introduit par Rosenblatt en 1958 [5]. Ce modèle, bien que prometteur, a rapidement montré ses limites. Il ne peut pas résoudre les problèmes non linéairement séparables, comme le montre l'exemple du XOR. Cette limitation a conduit au développement des réseaux de neurones multicouches, dont la capacité d'approximation est théoriquement illimitée, comme l'établit le théorème d'approximation universelle.

Cet article présente ces fondements en détail. Dans un premier temps, nous présentons le neurone formel, les fonctions d'activation, le perceptron et ses limites à travers le problème XOR. Ensuite, nous décrivons l'architecture des réseaux multicouches et la propagation avant, puis nous détaillons l'algorithme de rétropropagation et les méthodes d'optimisation associées. Nous abordons ensuite les problèmes pratiques rencontrés lors de l'entraînement et finalement nous concluons et ouvrons des perspectives.



2 Le perceptron : du neurone formel à ses limites

Un *neurone formel* est l'unité de base d'un réseau de neurones. Il prend en entrée un vecteur $\mathbf{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, associe à chaque entrée x_i un poids $w_i \in \mathbb{R}$, et produit une sortie scalaire. Le calcul se déroule en deux étapes. On calcule d'abord une combinaison linéaire des entrées pondérées à laquelle on ajoute un biais $b \in \mathbb{R}$, puis on applique une fonction d'activation σ . La sortie y du neurone est :

$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

où $\mathbf{w} = (w_1, \dots, w_n)^\top \in \mathbb{R}^n$ est le vecteur des poids.

Fonctions d'activation

La fonction d'activation permet au neurone de produire une sortie non linéaire. Sans elle, le neurone se réduirait à une simple combinaison linéaire de ses entrées, incapable de modéliser des relations complexes [1]. Plusieurs fonctions d'activation sont couramment utilisées.

- La *fonction sigmoïde* $\sigma : \mathbb{R} \rightarrow]0, 1[$ est définie par :

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Elle compresse toute valeur réelle dans l'intervalle $]0, 1[$, ce qui la rend interprétable comme une probabilité. Elle souffre cependant du problème de *gradient évanescent* [1]. Pour des valeurs de z très grandes ou très petites, sa dérivée tend vers zéro et l'apprentissage ralentit.

- La *tangente hyperbolique* $\tanh : \mathbb{R} \rightarrow]-1, 1[$, définie par :

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

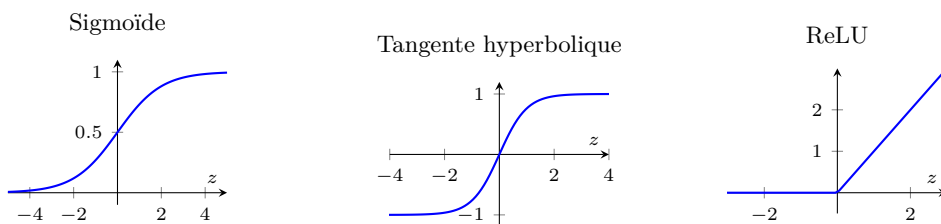
est centrée en zéro, ce qui accélère la convergence, mais souffre du même problème [1].

- La fonction *ReLU* (*Rectified Linear Unit*), aujourd'hui la plus utilisée dans les architectures profondes, est définie par :

$$\text{ReLU}(z) = \max(0, z)$$

Sa simplicité et le fait que sa dérivée vaille 1 pour $z > 0$ atténuent considérablement le problème de gradient évanescent. En contrepartie, les neurones pour lesquels $z \leq 0$ ont une dérivée nulle et cessent de contribuer à l'apprentissage, phénomène connu sous le nom de *dying ReLU*.

La Figure 1 illustre le comportement de ces trois fonctions.



■ **Figure 1** Graphes des trois fonctions d'activation présentées.

Le perceptron et sa règle d'apprentissage

Le *perceptron*, introduit par Rosenblatt [5], est le modèle le plus simple. Il s'agit d'un neurone formel unique dont la fonction d'activation est la fonction de seuil de Heaviside :

$$\sigma(z) = \begin{cases} 1 & \text{si } z \geq 0 \\ 0 & \text{sinon} \end{cases}$$

Étant donné un ensemble d'apprentissage $\{(\mathbf{x}^{(k)}, y^{(k)})\}_{k=1}^m$, où $\mathbf{x}^{(k)} \in \mathbb{R}^n$ est un vecteur d'entrée et $y^{(k)} \in \{0, 1\}$ est l'étiquette associée, le perceptron ajuste ses poids selon la règle suivante :

$$\mathbf{w} \leftarrow \mathbf{w} + \eta (y^{(k)} - \hat{y}^{(k)}) \mathbf{x}^{(k)}$$

où $\eta > 0$ est le *taux d'apprentissage* et $\hat{y}^{(k)}$ est la sortie prédite. Cette règle garantit la convergence lorsque les données sont linéairement séparables, comme l'établit le théorème de convergence du perceptron [5].

La limite du XOR

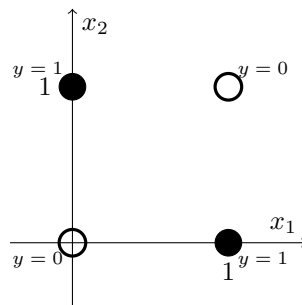
Le perceptron ne peut classer correctement que des données *linéairement séparables*, c'est-à-dire séparables par un hyperplan de $\mathbb{R}^n$. Le problème du *ou exclusif* (XOR) illustre parfaitement cette limite. La fonction XOR est définie sur $\{0, 1\}^2$ par le Tableau 1.

■ **Table 1** Table de vérité du XOR.

x_1	x_2	$x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

Proposition. *Il n'existe pas de perceptron à deux entrées, de fonction d'activation la fonction de Heaviside, qui réalise le « ou exclusif ».*

Nous sommes convaincus géométriquement qu'il n'existe pas de droite séparant les points de classe 0 des points de classe 1 dans le plan (x_1, x_2) , comme l'illustre la Figure 2. Nous le prouvons à présent par un calcul formel.



■ **Figure 2** Les quatre points du XOR : aucune droite ne peut séparer les cercles vides ($y = 0$) des points pleins ($y = 1$).

Démonstration. Nous raisonnons par l'absurde en supposant qu'un tel perceptron existe. Nous cherchons une contradiction. Notons $a, b, c \in \mathbb{R}$ ses coefficients, de sorte que la sortie est

$H(ax_1 + bx_2 + c)$ où H est la fonction de Heaviside. Le perceptron produit 1 si et seulement si $ax_1 + bx_2 + c \geq 0$.

- Pour $(x_1, x_2) = (1, 0)$ (sortie XOR = 1), nous devons avoir $a(1) + b(0) + c \geq 0$, donc

$$a + c \geq 0. \quad (1)$$

- Pour $(x_1, x_2) = (0, 1)$ (sortie XOR = 1), nous devons avoir $a(0) + b(1) + c \geq 0$, ce qui donne

$$b + c \geq 0. \quad (2)$$

- Pour $(x_1, x_2) = (0, 0)$ (sortie XOR = 0), nous devons avoir $a(0) + b(0) + c < 0$, ce qui donne

$$c < 0. \quad (3)$$

- Pour $(x_1, x_2) = (1, 1)$ (sortie XOR = 0), nous devons avoir $a(1) + b(1) + c < 0$, ce qui donne

$$a + b + c < 0. \quad (4)$$

En additionnant (1) et (2), nous obtenons $a + b + 2c \geq 0$. Or, par (3), $c < 0$, donc $-c > 0$. En additionnant cette inégalité stricte à la précédente, nous obtenons $a + b + c > 0$, ce qui contredit (4). Il ne peut donc exister de tels réels a, b, c , ce qui achève la démonstration, d'après Bodin et Recher [6].

Cette impossibilité montre qu'un neurone seul ne suffit pas pour les problèmes non linéairement séparables. Pour aller plus loin, il faut empiler plusieurs couches de neurones. C'est précisément l'idée des réseaux de neurones multicouches, présentés dans la section suivante.

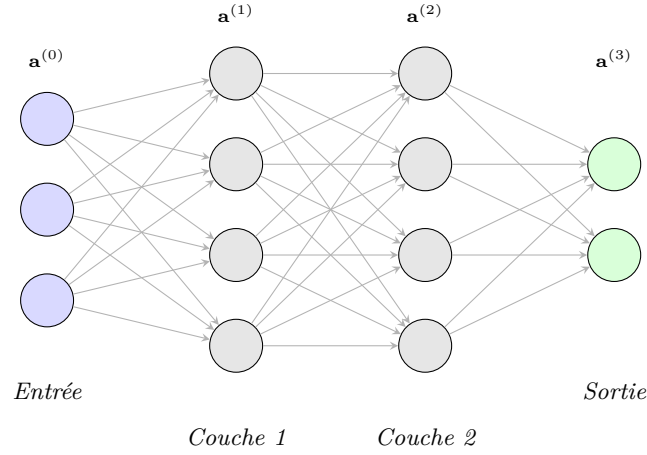
3 Réseaux multicouches et propagation avant

Un *réseau de neurones multicouches* (ou *MLP*, de l'anglais *Multi-Layer Perceptron*) généralise le perceptron en organisant des neurones en plusieurs couches successives. Dans cette section, nous décrivons l'architecture de ces réseaux, formalisons le calcul de leur sortie et rappelons le résultat théorique central qui justifie leur puissance expressive.

Architecture

Un MLP est composé de trois types de couches. La *couche d'entrée* reçoit le vecteur de données $\mathbf{x} \in \mathbb{R}^n$. Une ou plusieurs *couches cachées* transforment successivement cette représentation. Enfin, la *couche de sortie* produit la prédiction finale. On note L le nombre total de couches (sans compter la couche d'entrée), et n_l le nombre de neurones dans la couche l , pour $l = 1, \dots, L$.

La Figure 3 illustre l'architecture d'un MLP à deux couches cachées, où chaque neurone d'une couche est connecté à tous les neurones de la couche suivante.



■ **Figure 3** Architecture d'un MLP à deux couches cachées ($n_0 = 3, n_1 = n_2 = 4, n_3 = 2$). Chaque neurone est connecté à tous les neurones de la couche suivante.

Notation matricielle

Pour la couche l , on note $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ la matrice des poids, $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$ le vecteur des biais, σ la fonction d'activation appliquée composante par composante, $\mathbf{z}^{(l)}$ la *pré-activation* et $\mathbf{a}^{(l)}$ l'*activation* de la couche l . Le calcul se décompose en deux étapes :

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad (5)$$

$$\mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)}), \quad (6)$$

avec $\mathbf{a}^{(0)} = \mathbf{x}$ pour la couche d'entrée. La sortie du réseau est $\mathbf{a}^{(L)}$.

Propagation avant

La *propagation avant* (*forward pass*) consiste à calculer séquentiellement les équations (5) et (6) pour $l = 1, 2, \dots, L$, en partant de l'entrée $\mathbf{a}^{(0)} = \mathbf{x}$ et en progressant vers la couche de sortie. À chaque couche l , chaque neurone reçoit toutes les activations de la couche précédente, calcule leur combinaison linéaire affine, puis applique la fonction d'activation σ . Le résultat final $\mathbf{a}^{(L)}$ constitue la prédiction du réseau pour l'entrée $\mathbf{x}$.

Théorème d'approximation universelle

La puissance des MLP repose sur un résultat théorique fondamental. Cybenko [7] et Hornik [8] établissent indépendamment, à la fin des années 1980, le résultat suivant [6].

Théorème (Approximation universelle). *Toute fonction continue $f : [a, b]^n \rightarrow \mathbb{R}$ peut être approchée d'autant près que l'on veut par une fonction $F : [a, b]^n \rightarrow \mathbb{R}$ réalisée par un réseau de neurones à une couche cachée.*

Précisons le sens de « approcher d'autant près que l'on veut » : pour tout $\varepsilon > 0$, il existe un réseau F tel que :

$$\forall \mathbf{x} \in [a, b]^n, \quad |f(\mathbf{x}) - F(\mathbf{x})| < \varepsilon.$$

C'est l'*approximation uniforme* des fonctions. Plusieurs remarques s'imposent. Les hypothèses sont essentielles : f doit être continue et définie sur un compact $[a, b]^n$. Si l'une de

ces conditions venait à manquer, l'énoncé serait faux. Cybenko [7] montre en particulier que toute fonction présentant une discontinuité résiste à l'approximation uniforme par un nombre fini de neurones, quelle que soit la finesse du réseau. Par ailleurs, ce théorème établit l'existence d'un tel réseau sans indiquer comment construire ses poids. C'est précisément l'objet de la section suivante.

4 Rétropropagation et descente de gradient

La propagation avant permet de calculer la sortie d'un réseau pour une entrée donnée, mais ne dit rien sur la manière d'ajuster les poids pour que cette sortie soit correcte. La *rétropropagation du gradient*, proposée par Rumelhart et al. [9], est précisément la descente de gradient appliquée aux réseaux de neurones. Elle repose sur trois ingrédients. Une fonction de coût quantifie l'écart entre prédiction et cible. La descente de gradient minimise cette fonction par rapport à l'ensemble des poids. La règle de dérivation en chaîne rend ce calcul efficace couche par couche.

Fonction de coût

Pour mesurer la qualité des prédictions du réseau, nous avons besoin d'une fonction qui attribue un score scalaire à chaque configuration de poids $(W, \mathbf{b})$. C'est la *fonction de coût* $\mathcal{L}(W, \mathbf{b})$. Elle doit être nulle lorsque les prédictions sont parfaites, et d'autant plus grande que les erreurs sont importantes. Nous notons $\{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^N$ le jeu d'apprentissage, où $\mathbf{x}^{(i)} \in \mathbb{R}^{n_0}$ est le vecteur d'entrée et $\mathbf{y}^{(i)} \in \mathbb{R}^{n_L}$ la sortie attendue pour le i -ème exemple. La sortie produite par le réseau pour cet exemple est $\mathbf{a}^{(L)}(\mathbf{x}^{(i)})$, c'est-à-dire l'activation de la couche de sortie L obtenue par propagation avant.

Pour chaque exemple i , on quantifie l'écart entre prédiction et cible par une *erreur locale*, définie comme le carré de la norme euclidienne de la différence :

$$\mathcal{L}_i(W, \mathbf{b}) = \left\| \mathbf{a}^{(L)}(\mathbf{x}^{(i)}) - \mathbf{y}^{(i)} \right\|^2 = \sum_{k=1}^{n_L} \left(a_k^{(L)}(\mathbf{x}^{(i)}) - y_k^{(i)} \right)^2.$$

La *fonction de coût* globale, notée $\mathcal{L}(W, \mathbf{b})$, est la moyenne de ces erreurs locales sur l'ensemble des N exemples d'entraînement :

$$\mathcal{L}(W, \mathbf{b}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_i(W, \mathbf{b}) = \frac{1}{N} \sum_{i=1}^N \left\| \mathbf{a}^{(L)}(\mathbf{x}^{(i)}) - \mathbf{y}^{(i)} \right\|^2. \quad (7)$$

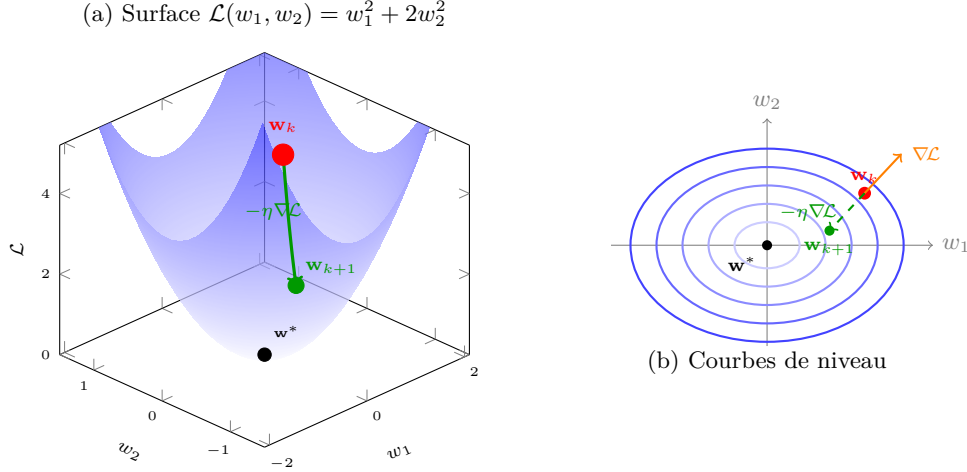
Cette formule, connue sous le nom d'*erreur quadratique moyenne* (MSE), vaut toujours $\mathcal{L} \geq 0$, et $\mathcal{L} = 0$ si et seulement si la prédiction est parfaite pour toutes les données [6]. Pour les problèmes de classification, Goodfellow et al. [1] recommandent la *cross-entropie*, qui pénalise davantage les prédictions erronées. Dans tous les cas, l'objectif de l'apprentissage est de trouver les poids $(W, \mathbf{b})$ qui minimisent $\mathcal{L}$.

Le gradient de la fonction de coût

Pour minimiser $\mathcal{L}$, il faut connaître, en chaque point de l'espace des poids, la direction dans laquelle $\mathcal{L}$ décroît le plus. Cette information est fournie par le *gradient*. Étant donné l'ensemble des poids $\mathbf{w} = (w_1, \dots, w_p)$, le gradient de $\mathcal{L}$ est le vecteur des dérivées partielles :

$$\nabla_{\mathbf{w}} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial w_1}, \dots, \frac{\partial \mathcal{L}}{\partial w_p} \right).$$

Il généralise la notion de dérivée aux fonctions de plusieurs variables. En dimension 1 (un seul poids w), il se réduit à la pente de la courbe $\mathcal{L}(w)$: si $\frac{d\mathcal{L}}{dw} > 0$, la fonction croît en w ; si $\frac{d\mathcal{L}}{dw} < 0$, elle décroît. En dimension 2 (deux poids w_1, w_2), le gradient est un vecteur du plan qui pointe perpendiculairement aux courbes de niveau dans la direction de la plus forte pente ascendante. Au voisinage d'un minimum, le gradient tend vers zéro, ce qui constitue le critère d'arrêt naturel de l'algorithme [6]. La Figure 4 illustre ces deux situations.



■ **Figure 4** Illustration de la descente de gradient pour $\mathcal{L}(w_1, w_2) = w_1^2 + 2w_2^2$. (a) Vue en surface 3D : la fonction forme un bol ; le pas $-\eta \nabla \mathcal{L}$ (flèche verte) déplace $\mathbf{w}_k$ vers le fond $\mathbf{w}^*$. (b) Vue de dessus (courbes de niveau) : le gradient (orange) pointe perpendiculairement aux ellipses dans la direction de plus forte montée ; son opposé ramène vers le minimum.

Descente de gradient

L'intuition de la descente de gradient est celle d'une goutte d'eau sur une colline qui suit la pente et s'arrête dans un creux [6]. Pour formaliser cela, on représente l'ensemble des poids du réseau, c'est-à-dire toutes les matrices $\mathbf{W}^{(l)}$ et tous les biais $\mathbf{b}^{(l)}$ empilés, par un seul vecteur $\mathbf{w}_k$, où $k = 0, 1, 2, \dots$ est un *compteur d'itérations* sans lien avec les indices de couches. On part d'un point initial $\mathbf{w}_0$ tiré au hasard, puis on itère la mise à jour :

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_k), \quad (8)$$

où $\eta > 0$ est le *taux d'apprentissage*. Le signe moins traduit le déplacement dans la direction de plus forte *diminution* de $\mathcal{L}$, car le gradient pointe vers la montée et son opposé vers la descente. Si η est trop grand, l'algorithme oscille autour du minimum sans converger ; s'il est trop petit, la convergence devient très lente [6].

Rétropropagation

Pour appliquer la règle (8), il faut calculer concrètement $\nabla_{\mathbf{w}} \mathcal{L}$, c'est-à-dire les gradients de $\mathcal{L}$ par rapport aux poids de *chaque* couche. La *rétropropagation* [9] y parvient en appliquant la règle de dérivation en chaîne couche par couche, en remontant de la sortie vers l'entrée. On définit pour chaque couche l le vecteur d'*erreur rétropropagée* :

$$\delta^{(l)} = \nabla_{\mathbf{z}^{(l)}} \mathcal{L}, \quad (9)$$

qui mesure la sensibilité de $\mathcal{L}$ aux pré-activations de la couche l . À la couche de sortie, la règle de dérivation composée appliquée à la MSE donne directement :

$$\delta^{(L)} = 2(\mathbf{a}^{(L)} - \mathbf{y}) \odot \sigma'(\mathbf{z}^{(L)}),$$

où $\odot$ désigne le *produit de Hadamard*, c'est-à-dire le produit terme à terme défini par $(\mathbf{u} \odot \mathbf{v})_j = u_j v_j$. Ce produit apparaît naturellement car la règle de la chaîne combine deux vecteurs, le gradient de $\mathcal{L}$ par rapport aux activations et la dérivée de l'activation par rapport aux pré-activations, que l'on ne peut ni additionner ni multiplier matriciellement. On remonte ensuite vers les couches précédentes grâce à la relation de récurrence

$$\delta^{(l)} = \left(\left(W^{(l+1)} \right)^\top \delta^{(l+1)} \right) \odot \sigma'(\mathbf{z}^{(l)}), \quad (10)$$

qui exprime que l'erreur de la couche l est obtenue en rétropropageant l'erreur de la couche $l+1$ via les poids $W^{(l+1)}$, puis en la modulant neurone par neurone par la pente de l'activation locale [1].

Mise à jour des poids

Une fois les $\delta^{(l)}$ calculés pour toutes les couches ($l = L, L-1, \dots, 1$), la règle de dérivation en chaîne fournit les gradients couche par couche :

$$\nabla_{W^{(l)}} \mathcal{L} = \delta^{(l)} \left(\mathbf{a}^{(l-1)} \right)^\top, \quad (11)$$

$$\nabla_{\mathbf{b}^{(l)}} \mathcal{L} = \delta^{(l)}. \quad (12)$$

La règle abstraite (8) se traduit alors, pour chaque couche l , par les mises à jour

$$W^{(l)} \leftarrow W^{(l)} - \eta \delta^{(l)} \left(\mathbf{a}^{(l-1)} \right)^\top, \quad \mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \delta^{(l)}. \quad (13)$$

L'algorithme complet alterne donc une propagation avant qui calcule tous les $\mathbf{z}^{(l)}$ et $\mathbf{a}^{(l)}$, une rétropropagation qui calcule les $\delta^{(l)}$ de $l = L$ jusqu'à $l = 1$, puis la mise à jour simultanée de tous les poids et biais du réseau.

Variantes de la descente de gradient

La fonction de coût (7) est une somme de N erreurs locales $\mathcal{L}_i$. La descente de gradient *batch* calcule le gradient exact $\nabla \mathcal{L} = \frac{1}{N} \sum_i \nabla \mathcal{L}_i$ en utilisant toutes les données à chaque itération, ce qui devient trop coûteux pour de grands N . La *descente de gradient stochastique* (SGD) contourne ce problème en approximant le gradient par celui d'une seule erreur locale $\nabla \mathcal{L}_i$ à chaque itération, ce qui réduit fortement le coût de calcul mais introduit du bruit [6]. La variante *mini-batch* trouve un équilibre en divisant les données en lots de taille K et en effectuant une mise à jour par lot. Un passage complet sur l'ensemble des données constitue une *époque* et plusieurs époques sont en général nécessaires pour converger. L'optimiseur *Adam*, proposé par Kingma et Ba [10], va encore plus loin en adaptant le taux d'apprentissage séparément pour chaque poids, ce qui accélère la convergence et réduit les oscillations.

Ces avancées algorithmiques ne garantissent cependant pas un entraînement sans difficulté : plusieurs problèmes pratiques surgissent lors de la mise en œuvre, que nous examinons dans la section suivante.

5 Problèmes pratiques et solutions

La théorie garantit qu'un bon réseau existe, mais l'entraînement en pratique soulève plusieurs difficultés. Nous présentons ici les problèmes les plus courants et les solutions généralement utilisées.

Surapprentissage et régularisation

Le *surapprentissage* (*overfitting*) se produit lorsque le réseau modélise trop fidèlement les données d'entraînement, au détriment de sa capacité à généraliser à de nouvelles données. Il se manifeste par un écart croissant entre l'erreur sur les données d'entraînement et l'erreur sur un ensemble de validation [1].

La *régularisation* L_2 (ou *weight decay*) pénalise les poids de grande amplitude en ajoutant le terme $\frac{\lambda}{2} \|W\|^2$ à la fonction de coût (7), ce qui contraint le réseau à préférer des solutions plus simples [1].

Le *Dropout*, introduit par Srivastava et al. [11], consiste à désactiver aléatoirement une fraction p des neurones à chaque itération d'entraînement. Cette technique force le réseau à apprendre des représentations redondantes, ce qui améliore sa robustesse.

Initialisation des poids

L'initialisation des poids influence fortement la convergence de l'entraînement. Une initialisation à zéro est inefficace car tous les neurones d'une même couche calculent le même gradient et évoluent de manière identique. L'initialisation de *Glorot* (ou *Xavier*), proposée par Glorot et Bengio [12], choisit les poids de façon à préserver la variance des activations d'une couche à l'autre, ce qui stabilise l'entraînement avec des fonctions d'activation symétriques. Pour les réseaux utilisant ReLU, l'initialisation de *He* [2] adapte ce principe en tenant compte du fait que ReLU annule la moitié des activations en moyenne.

Normalisation par lot

La *normalisation par lot* (*Batch Normalization*), introduite par Ioffe et Szegedy [13], normalise les pré-activations $\mathbf{z}^{(l)}$ au sein de chaque mini-lot à chaque étape de l'entraînement, de sorte que leur distribution reste centrée et de variance unitaire. Cette opération réduit la sensibilité à l'initialisation des poids, permet d'utiliser des taux d'apprentissage plus élevés et atténue le problème du gradient évanescent. Elle introduit deux paramètres apprenables par couche, une échelle et un biais, qui lui permettent de restaurer la capacité expressive du réseau si nécessaire.

Maîtrisées conjointement, ces techniques constituent le socle de l'entraînement moderne des réseaux de neurones profonds, dont nous tirons maintenant les principales conclusions.

6 Conclusion et perspectives

Cet article a présenté les fondements mathématiques des réseaux de neurones multicouches, depuis le neurone formel jusqu'aux méthodes d'entraînement modernes. Nous avons montré que le perceptron, bien que simple, échoue sur les problèmes non linéairement séparables. L'empilement de couches, formalisé par la notation matricielle des équations (5) et (6), surmonte cette limite et confère aux MLP la capacité d'approximation universelle

établie par Cybenko [7] et Hornik [8]. L'algorithme de rétropropagation, décrit par les équations (10) et (13), permet d'entraîner ces réseaux efficacement par descente de gradient [9].

Les MLP présentent cependant des limites importantes. Ils ne tiennent pas compte de la structure des données. Chaque entrée est traitée de manière indépendante, sans notion de voisinage entre les pixels d'une image ou entre les mots d'une phrase. Ces limitations ont motivé le développement de nouvelles architectures. Les *réseaux de neurones convolutifs* (CNN) exploitent la structure locale des images par des opérations de convolution. Les *réseaux récurrents* (RNN) traitent les séquences en maintenant un état caché au fil du temps. Les *Transformers*, fondés sur le mécanisme d'auto-attention, permettent de capturer des dépendances à longue portée dans les données séquentielles et constituent aujourd'hui le socle des grands modèles de langage. Ces architectures avancées s'appuient sur les mêmes principes que ceux présentés ici : propagation avant, rétropropagation, descente de gradient.

Les réseaux de neurones présentent aussi une fragilité que leurs bons résultats ne font pas toujours apparaître. Moosavi-Dezfooli et al. [3] montrent qu'il existe de petites perturbations, imperceptibles à l'œil humain, qui suffisent à tromper ces réseaux sur plus de 85% des images naturelles. Ces perturbations sont les mêmes quelle que soit l'image, ce qui montre que la fragilité vient du réseau lui-même et non des données. Cela rappelle qu'un réseau qui obtient de bons résultats en test n'est pas nécessairement fiable, et que comprendre les mécanismes d'apprentissage reste une question ouverte.

Références

- 1 Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016. Disponible en ligne : <https://www.deeplearningbook.org>.
- 2 Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers : Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.
- 3 Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, Omar Fawzi, and Pascal Frossard. Universal adversarial perturbations. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017. arXiv :1610.08401.
- 4 Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5 :115–133, 1943.
- 5 Frank Rosenblatt. The perceptron : A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6) :386–408, 1958.
- 6 Arnaud Bodin and François Recher. *Deepmath : Mathématiques pour l'apprentissage profond*. Exo7, 2021. Disponible en ligne : <http://exo7.emath.fr>.
- 7 George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4) :303–314, 1989.
- 8 Kurt Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2) :251–257, 1991.
- 9 David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323 :533–536, 1986.
- 10 Diederik P. Kingma and Jimmy Ba. Adam : A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.
- 11 Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout : A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15 :1929–1958, 2014.

- 12 Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256, 2010.
- 13 Sergey Ioffe and Christian Szegedy. Batch normalization : Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015.